22MAT230 MATHEMATICS FOR COMPUTING 4

***Batch : AIE -C***

***Team 7***

---


**G Prajwal Priyadarshan - CB.SC.U4AIE24214**

**Kabilan K - CB.SC.U4AIE24224**

**Kishore B - CB.SC.U4AIE24227**

**Rahul L S - CB.SC.U4AIE24248**


---

# LLM Guided Regularization of Pseudoinverse for Ill-Posed Signal Reconstruction

**Table of Contents**


Problem Statement....1
What Are ill Posed Inverse Problems?....2
Well Posed vs ill Posed Problems....2
  Example....2
  Singular Value Spectrum....3
  Stability Check....3
  Condition Number via Singular Value....4
Methodology....4
  Signal Generation....5
  Forward Models....6
  Noise Models....8
Reconstruction Methods....9
  PINV....10
  TSVD....10
  Tikhonov....12
  NIST....13
  FNIST....15
  Evaluation Metrics....17
Model Comparison....17
LLM Parameter selection....18


## Problem Statement

## What Are ill Posed Inverse Problems?

A problem Ax = b is WELL POSED if:

1) Solution exists

2) Solution is unique

3) Solution depends continuously on data

We study conditioning using singular values and condition number.

## Well Posed vs ill Posed Problems

### Example

1) Full Rank Matrix

```
clc, clearvars
n = 100;
A1 = randn(n);              % Random Full rank matrix

[U1, S1, V1] = svd(A1);
singular_vals_A1 = diag(S1);

% Condition number
cond_A1 = cond(A1);
disp('Condtion number of Well Posed Matrix')
```

```
Condtion number of Well Posed Matrix
```

```
disp(cond_A1)
```

```
   5.3544e+03
```

2) ill Posed Matrix

```
function A = rankdeficient_operator(n, r)
    [U,~,V] = svd(randn(n), 'econ');   % random orthogonal matrices
    s = [linspace(1,0.1,r) zeros(1,n-r)];   % r non-zero singular values
    A = U * diag(s) * V';   % construct matrix
end
A2 = rank_deficient_operator(n, 12);
[U2, S2, V2] = svd(A2);
singular_vals_A2 = diag(S2);

% Condition number
cond_A2 = cond(A2);

disp('Condition number of Ill posed Matrix:')
```

```
Condition number of Ill posed Matrix:
```

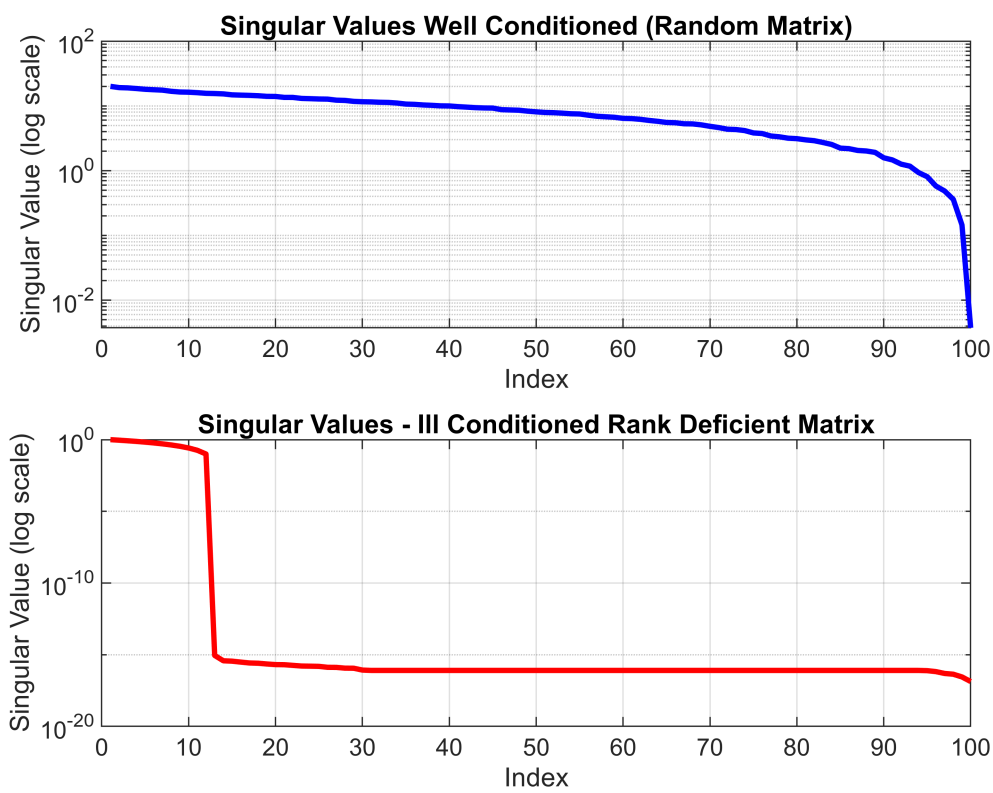
```
disp(cond_A2)
```

```
3.1797e+18
```

## Singular Value Spectrum

```
figure;
subplot(2,1,1)
semilogy(singular_vals_A1, 'b', 'LineWidth', 2);
grid on;
title('Singular Values Well Conditioned (Random Matrix)');
xlabel('Index');
ylabel('Singular Value (log scale)');

subplot(2,1,2)
semilogy(singular_vals_A2, 'r', 'LineWidth', 2);
grid on;
title('Singular Values - Ill Conditioned Rank Deficient Matrix');
xlabel('Index');
ylabel('Singular Value (log scale)');
```




## Stability Check

```
b1 = randn(n,1);
b1_noisy = b1 + 1e-6*randn(n,1);

x1_original = A1 \ b1;
x1_noisy    = A1 \ b1_noisy;
```

```
difference_well = norm(x1_original - x1_noisy);

x2_original = A2 \ b1;
```

```
Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND =  3.154765e-20.
```

```
x2_noisy    = A2 \ b1_noisy;
```

```
Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND =  3.154765e-20.
```

```
difference_ill = norm(x2_original - x2_noisy);

disp('Difference for Well-Conditioned Matrix:')
```

```
Difference for Well-Conditioned Matrix:
```

```
disp(difference_well)
```

```
   5.6937e-05
```

```

disp('Difference for Ill-Conditioned Matrix:')
```

```
Difference for Ill-Conditioned Matrix:
```

```
disp(difference_ill)
```

```
   4.1918e+12
```

### Condition Number via Singular Value

```
cond_manual_A1 = max(singular_vals_A1) / min(singular_vals_A1);
cond_manual_A2 = max(singular_vals_A2) / min(singular_vals_A2);

disp('Manual Condition Number (Random):')
```

```
Manual Condition Number (Random):
```

```
disp(cond_manual_A1)
```

```
   5.3544e+03
```

```


disp('Manual Condition Number (Hilbert):')
```

```
Manual Condition Number (Hilbert):
```

```
disp(cond_manual_A2)
```

```
   7.4082e+16
```

## Methodology

This study investigates inverse problem behavior under different numerical conditions. First, **synthetic signals** are generated to create controlled experimental data. A **forward model** is applied to simulate the measurement process, and **noise models** are introduced to represent real-world uncertainties. The **condition number** of the system matrix is analyzed to identify well-conditioned and ill-conditioned cases. Different **reconstruction methods** are then used to recover the original signal from noisy observations, and their performance is evaluated using suitable **evaluation metrics**.

## Signal Generation

```
function y = sinusoid(t)
    y = sin(2*pi*t) + 0.5*sin(6*pi*t);
end
```

```
function y = multisine(t, freqs, amps)
    if nargin < 2
        freqs = [2 5 9];
    end
    if nargin < 3
        amps = [1.0 0.6 0.3];
    end

    y = zeros(size(t));
    for k = 1:length(freqs)
        y = y + amps(k) * sin(2*pi*freqs(k)*t);
    end
end
```

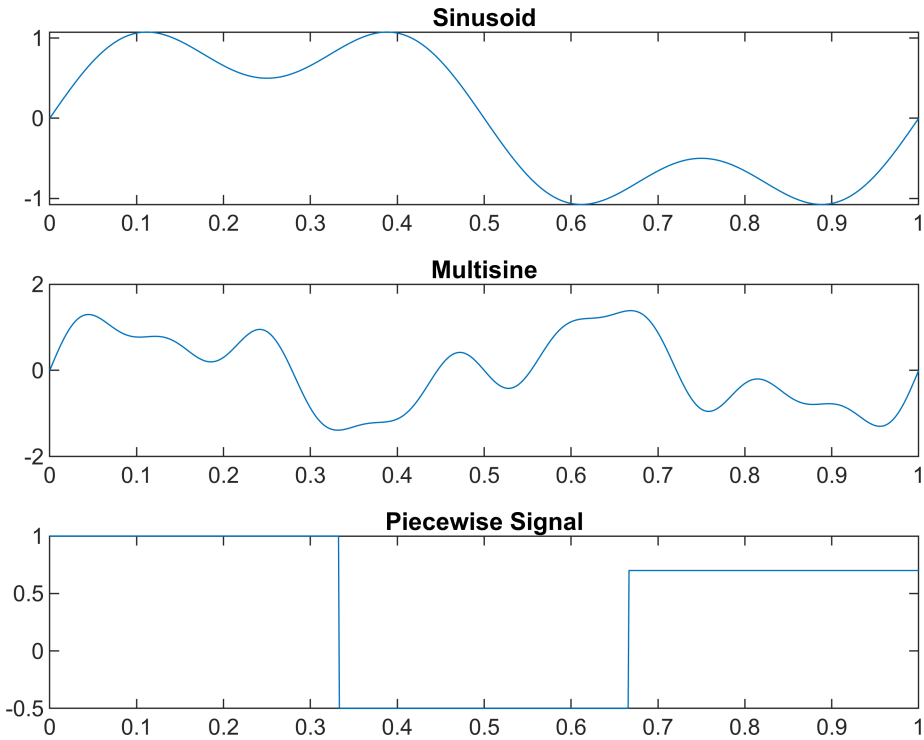
```
function y = piecewise_signal(t)
    y = zeros(size(t));
    N = length(t);
    thirds = floor(N/3);

    y(1:thirds) = 1.0;
    y(thirds+1:2*thirds) = -0.5;
    y(2*thirds+1:end) = 0.7;
end
```

```
t = linspace(0, 1, 1000);

y1 = sinusoid(t);
y2 = multisine(t);
y3 = piecewise_signal(t);

figure;
subplot(3,1,1); plot(t, y1); title('Sinusoid');
subplot(3,1,2); plot(t, y2); title('Multisine');
subplot(3,1,3); plot(t, y3); title('Piecewise Signal');
```




## Forward Models

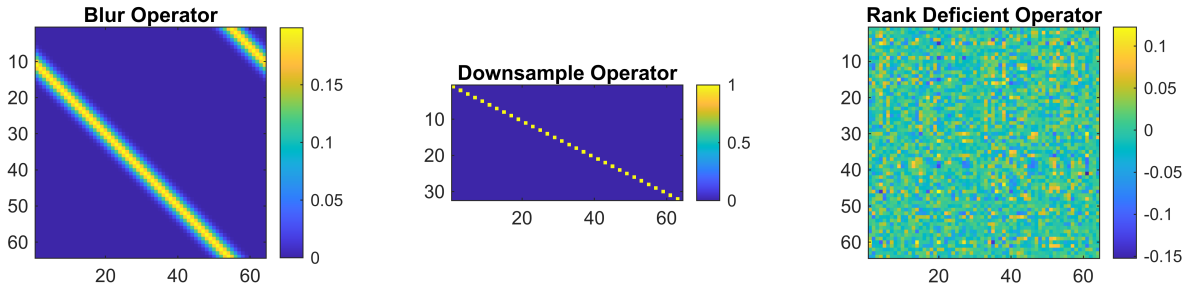
```
n = 100;

A_blur = blur_operator(n, 2.0);
A_down = downsample_operator(n, 2);
A_rank = rankdeficient_operator(n, 12);

figure('Position',[100 100 1000 300])
subplot(1,3,1); imagesc(A_blur); axis image;
title('Blur Operator'); colorbar;

subplot(1,3,2); imagesc(A_down); axis image;
title('Downsample Operator'); colorbar;

subplot(1,3,3); imagesc(A_rank); axis image;
title('Rank Deficient Operator'); colorbar;
```




```
function A = blur_operator(n, sigma, kernel_radius)
% BLUR_OPERATOR  Circulant Gaussian blur matrix

    if nargin < 3
        kernel_radius = 10;
    end

    ksize = 2 * kernel_radius + 1;
    k = gaussian_kernel(ksize, sigma);

    A = zeros(n, n);

    for i = 1:n
        for j = 1:n
            shift = mod(i - j, n);
            if shift < ksize
                A(i, j) = k(shift + 1);
            end
        end
    end
end
```

```
function A = downsample_operator(n, factor)
% DOWNSAMPLE_OPERATOR  Downsampling matrix

    if nargin < 2
        factor = 2;
    end

    m = floor(n / factor);
    A = zeros(m, n);

    for i = 1:m
        A(i, (i-1)*factor + 1) = 1.0;
    end
```

```
end
```

```
function A = rank_deficient_operator(n, r)
    [U,~,V] = svd(randn(n), 'econ');   % random orthogonal matrices
    s = [linspace(1,0.1,r) zeros(1,n-r)];   % r non-zero singular values
    A = U * diag(s) * V';   % construct matrix
end
```

```
function k = gaussian_kernel(sz, sigma)
% GAUSSIAN_KERNEL  1D Gaussian kernel

    idx = -(floor(sz/2)) : floor(sz/2);
    k = exp(-0.5 * (idx / sigma).^2);
    k = k / sum(k);
end
```

## Noise Models

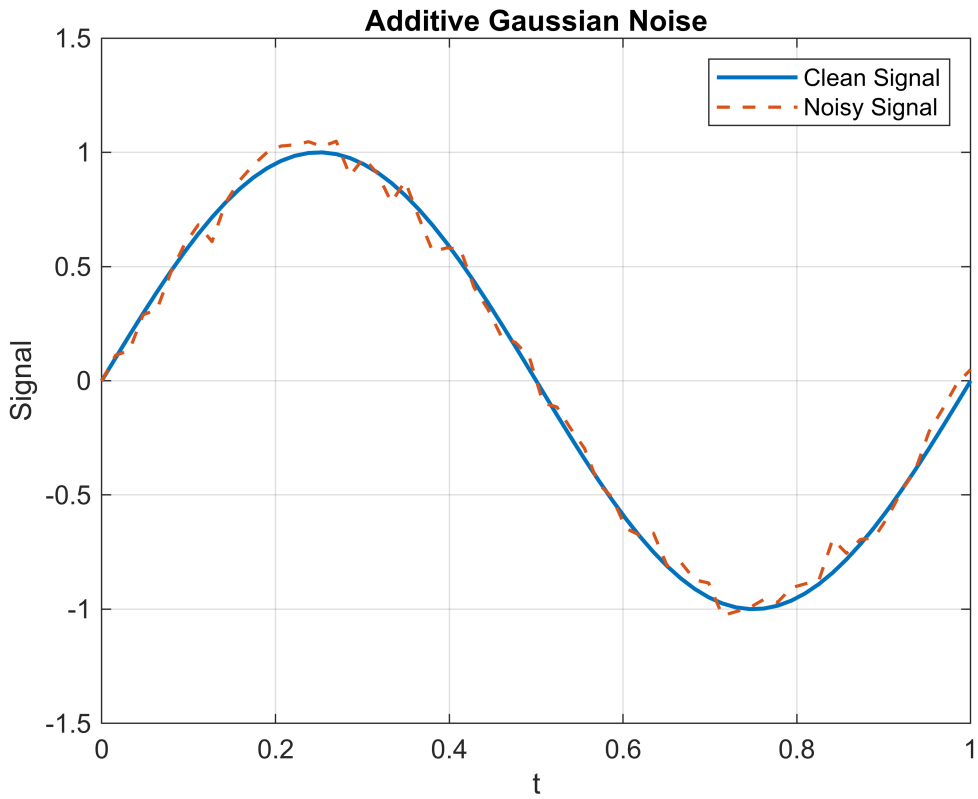
```
n = 100;
t = linspace(0,1,n)';
x = sin(2*pi*t);

sigma = 0.05;   % noise level

[y_noisy, noise] = add_gaussian_noise(x, sigma);

figure
plot(t,x,'LineWidth',1.5)
hold on
plot(t,y_noisy,'--','LineWidth',1.2)

legend('Clean Signal','Noisy Signal')
title('Additive Gaussian Noise')
xlabel('t')
ylabel('Signal')
grid on
```




```
function [y_noisy, noise] = add_gaussian_noise(y, sigma, rng)

    if nargin < 3 || isempty(rng)
        noise = sigma * randn(size(y));
    else
        noise = sigma * randn(rng, size(y));
    end

    y_noisy = y + noise;

end
```

## Reconstruction Methods

```
n = 100;                % signal length
noise_level = 0.08;     % noise standard deviation
blur_sigma = 2.5;       % blur kernel width

% Generate true signal
t = linspace(0,1,n);
x_true = sin(2*pi*t)' + 0.5*sin(6*pi*t)';

% Create forward operator (blur matrix)
A = blur_operator(n, blur_sigma, 10);
```

```
% Compute condition number
cond_num = cond(A);

fprintf('Forward operator matrix A: shape = %d x %d\n', size(A,1), size(A,2));
```

```
Forward operator matrix A: shape = 100 x 100
```

```
fprintf('Condition number (using diagnostics): κ(A) = %.2e\n', cond_num);
```

```
Condition number (using diagnostics): κ(A) = 1.94e+06
```

```


% Forward model
y_clean = A * x_true;

% Add Gaussian noise
[y_noisy, noise_vec] = add_gaussian_noise(y_clean, noise_level);

%From this we will use A and y_noisy for reconstruction
```

## PINV

```
function x = pseudoinverse_reconstruct(A, y_noisy)
    % Reconstruct signal using pseudoinverse
    % Inputs:
    % A - Forward operator matrix (n x m)
    % y_noisy - Noisy measurement vector (n x 1)

    % Output:
    % x - Reconstructed signal (m x 1)

    x = pinv(A) * y_noisy;
end

% Reconstruct
x_pinv = pseudoinverse_reconstruct(A, y_noisy);

% Compute reconstruction errors
mse_val = mse_error(x_true, x_pinv);
rel_error = relative_error(x_true, x_pinv);

fprintf('Relative Error: %.4f\n', rel_error);
```

```
Relative Error: 27107.9725
```

```
fprintf('MSE: %.6f\n', mse_val);
```

```
MSE: 454683593.547804
```

## TSVD

```
function x = tsvd_reconstruct(A, y, k)
    % TSVD_RECONSTRUCT  Truncated SVD reconstruction
    % Inputs:
    % A - Forward operator matrix
    % y - noisy observation
    % k - truncation parameter
    % Output:
    % x - reconstructed signal

    [U,S,V] = svd(A,'econ');

    % Truncate SVD components
    Uk = U(:,1:k);
    Sk = S(1:k,1:k);
    Vk = V(:,1:k);

    % TSVD solution
    x = Vk * (Sk \ (Uk' * y));
end
```

```
% Parameter sweep for TSVD
step = max(1, floor(n/50));
ks = 1:step:n-1;

num_k = length(ks);

mse_vals = zeros(num_k,1);
rel_vals = zeros(num_k,1);

x_store = cell(num_k,1);

for i = 1:num_k

    k = ks(i);

    % TSVD reconstruction
    x_tsvd = tsvd_reconstruct(A, y_noisy, k);

    % Metrics
    mse_vals(i) = mse_error(x_true, x_tsvd);
    rel_vals(i) = relative_error(x_true, x_tsvd);

    x_store{i} = x_tsvd;

end

% Find optimal k
[best_mse_tsvd, best_idx] = min(mse_vals);
best_k = ks(best_idx);
```

```
best_rel_tsvd = rel_vals(best_idx);
x_best_tsvd = x_store{best_idx};

fprintf('\nTSVD Parameter Sweep Results:\n');
```

```
TSVD Parameter Sweep Results:
```

```
fprintf('k range: %d to %d\n', ks(1), ks(end));
```

```
k range: 1 to 99
```

```
fprintf('Optimal k: %d\n', best_k);
```

```
Optimal k: 7
```

```
fprintf('Optimal MSE: %.6f\n', best_mse_tsvd);
```

```
Optimal MSE: 0.000948
```

```
fprintf('Optimal Relative Error: %.6f\n', best_rel_tsvd);
```

```
Optimal Relative Error: 0.039142
```

## Tikhonov

```
function x_tik = tikhonov_reconstruct(A, y, lambda)
% Tikhonov reconstruction
% Solves: min ||Ax - y||^2 + lambda ||x||^2

% Inputs:
% A - Forward operator matrix
% y - Observed noisy data
% lambda - Regularization parameter

% Output:
% x_tik - Reconstructed signal

[m, n] = size(A);

% Identity matrix
I = eye(n);

% Tikhonov solution
x_tik = (A' * A + lambda * I) \ (A' * y);
end
```

```
lambdas = logspace(-6,2,50);

mse_vals = zeros(length(lambdas),1);
rel_err_vals = zeros(length(lambdas),1);

x_results = cell(length(lambdas),1);
```

```
for i = 1:length(lambdas)

    lam = lambdas(i);

    % Reconstruction
    x_tik = tikhonov_reconstruct(A, y_noisy, lam);

    % Metrics
    mse_vals(i) = mse_error(x_true, x_tik);
    rel_err_vals(i) = relative_error(x_true, x_tik);

    x_results{i} = x_tik;

end

% Find optimal lambda
[best_mse_tikh, best_idx] = min(mse_vals);

best_lambda = lambdas(best_idx);
best_rel_tikh = rel_err_vals(best_idx);
x_best_tikh = x_results{best_idx};

fprintf('Tikhonov Parameter Sweep Results:\n');
```

```
Tikhonov Parameter Sweep Results:
```

```
fprintf('Lambda range: %.2e to %.2e\n', lambdas(1), lambdas(end));
```

```
Lambda range: 1.00e-06 to 1.00e+02
```

```
fprintf('Optimal lambda: %.6e\n', best_lambda);
```

```
Optimal lambda: 7.906043e-02
```

```
fprintf('Optimal MSE: %.6f\n', best_mse_tikh);
```

```
Optimal MSE: 0.008548
```

```
fprintf('Optimal Relative Error: %.6f\n', best_rel_tikh);
```

```
Optimal Relative Error: 0.117539
```

## NIST

```
function [x_rec, residuals, rel_errors, stop_iter] = nsit_morozov(A, y, x_true,
delta, q, tau, max_iter)

% NSIT reconstruction using Morozov discrepancy principle
% A        : forward operator
% y        : noisy measurements
% x_true   : true signal (only for error evaluation)
% delta    : noise level
```

```
% q        : geometric decay factor (0<q<1)
% tau      : Morozov safety factor
% max_iter : maximum iterations

[m,n] = size(A);

x = zeros(n,1);

At = A';
AtA = At*A;

% Proper scaling of alpha
alpha0 = norm(A)^2;

residuals = zeros(max_iter,1);
rel_errors = zeros(max_iter,1);

for k = 1:max_iter

    % Geometric alpha schedule
    alpha = alpha0 * q^(k-1);

    % Compute residual
    r = y - A*x;

    % NSIT correction step
    correction = (AtA + alpha*eye(n)) \ (At*r);

    % Update solution
    x = x + correction;

    % Residual norm
    residuals(k) = norm(y - A*x);

    % Relative error (for evaluation)
    rel_errors(k) = relative_error(x_true, x);

    % Morozov stopping rule
    if residuals(k) <= tau * delta
        break
    end

end

x_rec = x;
stop_iter = k;

residuals = residuals(1:k);
rel_errors = rel_errors(1:k);
```

```
end
```

```
max_iter = 200;
q = 0.9;
tau = 1.01;

delta = norm(noise_vec);

[x_nsit, residuals, errors, stop_iter] = nsit_morozov(A, y_noisy, x_true, delta, q,
tau, max_iter);

nsit_best_mse = mse_error(x_true,x_nsit);
nsit_best_rel = errors(end);

fprintf('NSIT stopped at iteration %d\n', stop_iter);
```

```
NSIT stopped at iteration 5
```

```
fprintf('Mean Squared Error = %.6f\n', nsit_best_mse);
```

```
Mean Squared Error = 0.003386
```

```
fprintf('Relative Error = %.6f\n', nsit_best_rel);
```

```
Relative Error = 0.073978
```

## FNIST

```
function x_rec = fnist_reconstruction(A, y, alpha0, q, max_iter)
    [~,n] = size(A);
    x = zeros(n,1);
    At = A';
    AtA = At*A;

    for k = 1:max_iter
        alpha = alpha0 * q^(k-1);
        r = y - A*x;
        % First correction
        F1 = (AtA + alpha*eye(n)) \ (At*r);
        % Second correction (correct form)
        temp = (AtA + alpha^2*eye(n)) \ (At*r);
        F2 = alpha * ((AtA + alpha*eye(n)) \ temp);
        % Stable update
        x = x + 0.7*F1 + 0.3*F2;
    end
    x_rec = x;
end

alpha0 = 1;
```

```
q = 0.9;
max_iter = 10;

x_fnist = fnist_reconstruction(A, y_noisy, alpha0, q, max_iter);

% metrics
mse_val = mse_error(x_true, x_fnist);
rel_error = relative_error(x_true, x_fnist);

fprintf('\nFNIST Reconstruction Results:\n');
```

```
FNIST Reconstruction Results:
```

```
fprintf('Relative Error: %.6f\n', rel_error);
```

```
Relative Error: 0.124670
```

```
fprintf('MSE: %.6f\n', mse_val);
```

```
MSE: 0.009617
```

```
alpha_vals = logspace(-2,2,25);   % alpha0 values
q_vals = 0.85:0.01:0.99;           % decay factors
iter_vals = 5:30;                 % iterations

best_mse = inf;
best_rel = inf;
best_alpha = 0;
best_q = 0;
best_iter = 0;
best_x = zeros(size(x_true));

for a = 1:length(alpha_vals)
    for b = 1:length(q_vals)
        for c = 1:length(iter_vals)

            alpha0 = alpha_vals(a);
            q = q_vals(b);
            max_iter = iter_vals(c);

            x_fnist = fnist_reconstruction(A, y_noisy, alpha0, q, max_iter);

            mse_val = mse_error(x_true, x_fnist);
            rel_err = relative_error(x_true, x_fnist);

            if mse_val < best_mse
                best_mse = mse_val;
                best_rel = rel_err;
                best_alpha = alpha0;
                best_q = q;
                best_iter = max_iter;
```

```
                best_x = x_fnist;
            end
        end
    end
end

best_mse_fnist = best_mse;
best_rel_fnist = best_rel;
x_fnist_best = best_x;

fprintf("\nOptimal FNIST Parameters:\n");
fprintf("alpha0 = %.4f\n", best_alpha);
fprintf("q = %.3f\n", best_q);
fprintf("iterations = %d\n", best_iter);
fprintf("Best MSE = %.6f\n", best_mse_fnist);
fprintf("Best Relative Error = %.6f\n", best_rel_fnist);
```

### Evaluation Metrics

```
function err = relative_error(x_true, x_rec)
err = norm(x_true - x_rec) / norm(x_true);
end

function mse = mse_error(x_true, x_rec)
mse = mean((x_true - x_rec).^2);
end
```

## Model Comparison

```
methods = {'Pseudoinverse','TSVD','Tikhonov','NSIT','FNIST'};

rel_errors = [
    relative_error(x_true, x_pinv);
    best_rel_tsvd;
    best_rel_tikh;
    nsit_best_rel;
    best_rel_fnist
];

mses = [
    mse_error(x_true, x_pinv);
    best_mse_tsvd;
    best_mse_tikh;
    nsit_best_mse;
    best_mse_fnist
];

fprintf('\n=====================================\n');
```

```
=====================================
```

```
fprintf('Reconstruction Method Comparison\n');
```

```
Reconstruction Method Comparison
```

```
fprintf('=====================================\n');
```

```
=====================================
```

```
for i = 1:length(methods)
    fprintf('%-12s | Rel Error: %.6f | MSE: %.6f\n',...
        methods{i}, rel_errors(i), mses(i));
end
```

```
Pseudoinverse | Rel Error: 27107.972469 | MSE: 454683593.547804
TSVD         | Rel Error: 0.039142 | MSE: 0.000948
Tikhonov     | Rel Error: 0.117539 | MSE: 0.008548
NSIT         | Rel Error: 0.073978 | MSE: 0.003386
FNIST        | Rel Error: 0.065126 | MSE: 0.002624
```

## LLM Parameter selection

```
% BUILD PROMPT FOR LLM

prompt = "";

prompt = prompt + "You are an expert in numerical linear algebra and inverse
problems." + newline;
prompt = prompt + "Your task is to recommend optimal regularization parameters for
solving an ill-posed inverse problem." + newline;
prompt = prompt + "We are solving the reconstruction problem Ax = y where A is
ill-conditioned and the measurements contain noise." + newline + newline;

prompt = prompt + "=====================================" + newline;
prompt = prompt + "PROBLEM DESCRIPTION" + newline;
prompt = prompt + "=====================================" + newline;

prompt = prompt + "Signal model:" + newline;
prompt = prompt + "- Signal type: multisine signal (sum of multiple sinusoids)" +
newline;
prompt = prompt + "- Signal length: " + num2str(n) + newline;
prompt = prompt + "- Sampling interval: uniform in [0,1]" + newline + newline;

prompt = prompt + "Forward model:" + newline;
prompt = prompt + "- Operator type: Gaussian blur operator" + newline;
prompt = prompt + "- Blur sigma: " + num2str(blur_sigma) + newline;
prompt = prompt + "- Matrix size: " + num2str(size(A,1)) + " x " +
num2str(size(A,2)) + newline;
prompt = prompt + "- Matrix rank: " + num2str(rank(A)) + newline;
prompt = prompt + "- Condition number: " + num2str(cond_num) + newline + newline;

prompt = prompt + "Noise model:" + newline;
```

```
prompt = prompt + "- Additive Gaussian noise" + newline;
prompt = prompt + "- Noise standard deviation: " + num2str(noise_level) + newline;
prompt = prompt + "- Noise norm: " + num2str(norm(noise_vec)) + newline + newline;

prompt = prompt + "======================================" + newline;
prompt = prompt + "CURRENT RECONSTRUCTION PERFORMANCE" + newline;
prompt = prompt + "======================================" + newline;

prompt = prompt + "Relative reconstruction errors:" + newline;
prompt = prompt + "- Pseudoinverse: " + num2str(rel_errors(1)) + newline;
prompt = prompt + "- TSVD: " + num2str(rel_errors(2)) + newline;
prompt = prompt + "- Tikhonov: " + num2str(rel_errors(3)) + newline;
prompt = prompt + "- NSIT: " + num2str(rel_errors(4)) + newline;
prompt = prompt + "- FNIST: " + num2str(rel_errors(5)) + newline + newline;

prompt = prompt + "Mean squared errors:" + newline;
prompt = prompt + "- Pseudoinverse: " + num2str(mses(1)) + newline;
prompt = prompt + "- TSVD: " + num2str(mses(2)) + newline;
prompt = prompt + "- Tikhonov: " + num2str(mses(3)) + newline;
prompt = prompt + "- NSIT: " + num2str(mses(4)) + newline;
prompt = prompt + "- FNIST: " + num2str(mses(5)) + newline + newline;

prompt = prompt + "======================================" + newline;
prompt = prompt + "OBJECTIVE" + newline;
prompt = prompt + "======================================" + newline;

prompt = prompt + "Select improved parameters that minimize reconstruction error
and improve stability." + newline;
prompt = prompt + "Consider the ill-conditioning of A and the noise level when
choosing parameters." + newline;
prompt = prompt + "Avoid parameter choices that cause instability or noise
amplification." + newline + newline;

prompt = prompt + "======================================" + newline;
prompt = prompt + "ALGORITHMS TO TUNE" + newline;
prompt = prompt + "======================================" + newline;

prompt = prompt + "1) TSVD: choose truncation parameter k" + newline;
prompt = prompt + "2) Tikhonov: choose regularization parameter lambda" + newline;
prompt = prompt + "3) NSIT: choose q, iterations, delta, tau" + newline;
prompt = prompt + "4) FNIST: choose alpha0, q, iterations" + newline + newline;

prompt = prompt + "======================================" + newline;
prompt = prompt + "PARAMETER CONSTRAINTS" + newline;
prompt = prompt + "======================================" + newline;

prompt = prompt + "- k must be between 5 and 30" + newline;
prompt = prompt + "- lambda between 1e-6 and 1" + newline;
prompt = prompt + "- alpha0 between 0.001 and 0.5" + newline;
prompt = prompt + "- q between 0.5 and 0.95" + newline;
```

```
prompt = prompt + "- iterations between 10 and 60" + newline;
prompt = prompt + "- tau between 1.0 and 1.2" + newline;
prompt = prompt + "- delta should be close to the noise norm" + newline + newline;

prompt = prompt + "=====================================" + newline;
prompt = prompt + "RESPONSE FORMAT" + newline;
prompt = prompt + "=====================================" + newline;

prompt = prompt + "Return ONLY valid JSON with no explanation." + newline;
prompt = prompt + "Use exactly this structure:" + newline;

prompt = prompt + '{' + newline;
prompt = prompt + ' "TSVD": {' + newline;
prompt = prompt + '   "k": 12' + newline;
prompt = prompt + ' },' + newline;
prompt = prompt + ' "Tikhonov": {' + newline;
prompt = prompt + '   "lambda": 0.015' + newline;
prompt = prompt + ' },' + newline;
prompt = prompt + ' "NSIT": {' + newline;
prompt = prompt + '   "q": 0.92,' + newline;
prompt = prompt + '   "iterations": 15,' + newline;
prompt = prompt + '   "delta": 0.1,' + newline;
prompt = prompt + '   "tau": 1.02' + newline;
prompt = prompt + ' },' + newline;
prompt = prompt + ' "FNIST": {' + newline;
prompt = prompt + '   "alpha0": 0.15,' + newline;
prompt = prompt + '   "q": 0.9,' + newline;
prompt = prompt + '   "iterations": 18' + newline;
prompt = prompt + ' }' + newline;
prompt = prompt + '}';
```

```
api_key = "AIzaSyCDPkmuUnzZpdr63Smj4g0eKHd8jrOMOO8";
params = get_llm_parameters_gemini(prompt, api_key);
k_llm = params.TSVD.k;

lambda_llm = params.Tikhonov.lambda;

q_nsit = params.NSIT.q;
iter_nsit = params.NSIT.iterations;
delta_nsit = params.NSIT.delta;
tau_nsit = params.NSIT.tau;

alpha_fnist = params.FNIST.alpha0;
q_fnist = params.FNIST.q;
iter_fnist = params.FNIST.iterations;

disp("LLM Suggested Parameters:")
```

```
LLM Suggested Parameters:
```

```
disp(params)
```

```
        TSVD: [1×1 struct]
    Tikhonov: [1×1 struct]
        NSIT: [1×1 struct]
       FNIST: [1×1 struct]
```

```
% RECONSTRUCTION WITH LLM PARAMETERS

% Tikhonov
x_rec_tikh_llm = tikhonov_reconstruct(A, y_noisy, lambda_llm);

% TSVD
x_rec_tsvd_llm = tsvd_reconstruct(A, y_noisy, k_llm);

% NSIT
[x_rec_nsit_llm,~,~,~] =
nsit_morozov(A,y_noisy,x_true,delta_nsit,q_nsit,tau_nsit,iter_nsit);

% FNIST
x_rec_fnist_llm = fnist_reconstruction( ...
    A, ...
    y_noisy, ...
    alpha_fnist, ...
    q_fnist, ...
    iter_fnist );

% COMPUTE ERRORS
rel_tsvd_llm  = relative_error(x_true, x_rec_tsvd_llm);
rel_nsit_llm  = relative_error(x_true, x_rec_nsit_llm);
rel_fnist_llm = relative_error(x_true, x_rec_fnist_llm);
rel_tikh_llm = relative_error(x_true, x_rec_tikh_llm);

mse_tsvd_llm  = mse_error(x_true, x_rec_tsvd_llm);
mse_nsit_llm  = mse_error(x_true, x_rec_nsit_llm);
mse_fnist_llm = mse_error(x_true, x_rec_fnist_llm);
mse_tikh_llm = mse_error(x_true, x_rec_tikh_llm);
```

```
%% ==========================================================
%% VISUALIZATION
%% ==========================================================

figure

subplot(3,2,1)
plot(t',x_true,'LineWidth',2)
hold on
plot(t',y_noisy,'--')
title("Original vs Observed")
```

```
legend("True","Observed")
grid on

subplot(3,2,2)
plot(t',x_true,'LineWidth',2)
hold on
plot(t',x_rec_tsvd_llm)
title("TSVD")
legend("True","TSVD","TSVD + LLM")
```

```
Warning: Ignoring extra legend entries.
```

```
grid on

subplot(3,2,3)
plot(t',x_true,'LineWidth',2)
hold on
plot(t',x_rec_tikh_llm)
title("Tikhonov")
legend("True","Tikhonov","Tikhonov + LLM")
```

```
Warning: Ignoring extra legend entries.
```

```
grid on

subplot(3,2,4)
plot(t',x_true,'LineWidth',2)
hold on
plot(t',x_rec_nsit_llm)
title("NSIT")
legend("True","NSIT","NSIT + LLM")
```

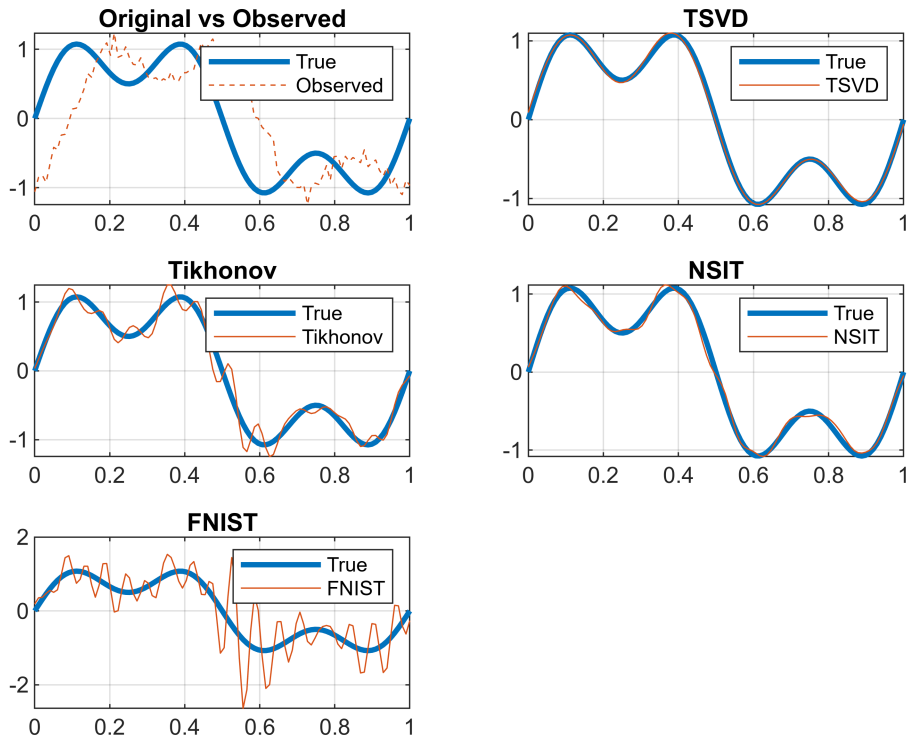
```
Warning: Ignoring extra legend entries.
```

```
grid on

subplot(3,2,5)
plot(t',x_true,'LineWidth',2)
hold on
plot(t',x_rec_fnist_llm)
title("FNIST")
legend("True","FNIST","FNIST + LLM")
```

```
Warning: Ignoring extra legend entries.
```

```
grid on
```




```
% PRINT FINAL RESULTS

fprintf("\n----- Reconstruction Errors -----\n")
```

```
----- Reconstruction Errors -----
```

```
fprintf("TSVD                : %.6f\n",best_rel_tsvd)
```

```
TSVD              : 0.039142
```

```
fprintf("TSVD + LLM          : %.6f\n",rel_tsvd_llm)
```

```
TSVD + LLM        : 0.041002
```

```
fprintf("Tikhonov            : %.6f\n",best_rel_tikh)
```

```
Tikhonov          : 0.117539
```

```
fprintf("Tikhonov + LLM      : %.6f\n",rel_tikh_llm)
```

```
Tikhonov + LLM    : 0.181156
```

```
fprintf("NSIT                : %.6f\n",nsit_best_rel)
```

```
NSIT              : 0.073978
```

```
fprintf("NSIT + LLM        : %.6f\n",rel_nsit_llm)
```

```
NSIT + LLM       : 0.073509
```

```
fprintf("FNIST             : %.6f\n",best_rel_fnist)
```

```
FNIST            : 0.065126
```

```
fprintf("FNIST + LLM       : %.6f\n",rel_fnist_llm)
```

```
FNIST + LLM      : 0.700339
```

```
function params = get_llm_parameters_gemini(prompt, api_key)

url = "https://generativelanguage.googleapis.com/v1/models/gemini-2.5-
flash:generateContent?key=" + api_key;
data = struct;
data.contents = {struct("parts",{struct("text",prompt)})};

options = weboptions( ...
    'MediaType','application/json', ...
    'ContentType','json', ...
    'Timeout',60);

response = webwrite(url,data,options);

text = response.candidates(1).content.parts(1).text;
% Remove markdown if present
text = erase(text,"```json");
text = erase(text,"```");

% Extract JSON safely
start_idx = find(text=='{',1,'first');
end_idx   = find(text=='}',1,'last');

json_text = text(start_idx:end_idx);

params = jsondecode(json_text);

end
```